

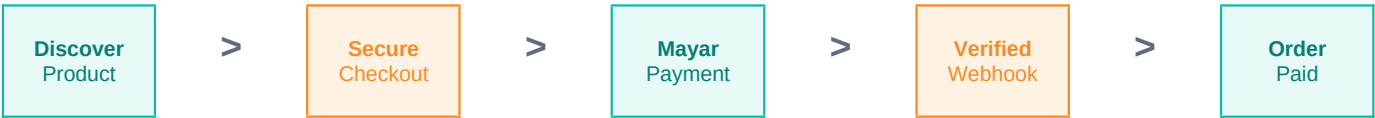
WEBINAR DEMO BLUEPRINT

WarungKit

Business Requirements Document & Technical Blueprint

Vibe Coding to Secure Payment Integration for UMKM

A production-aware webinar demo that shows how an AI-built UMKM storefront can safely create payment links, receive payment events, and protect customer data.



Document Control	Details
Document Status	Draft v1.0
Project Type	Webinar Demo / Production-aware MVP
Primary Audience	UMKM, creators, aspiring vibe coders, and developers
Target Duration	60 minutes live session
Prepared For	WarungKit Webinar Demo
Document Date	4 July 2026

Decision Statement

WarungKit will be built as a focused digital-product storefront. The public interface is intentionally simple; payment, database access, and secrets stay behind a backend API boundary.

Contents

Section	Content
1	Executive Summary
2	Business Context and Webinar Objective
3	Product Vision, Personas, and Scope
4	User Journey and Business Rules
5	Functional and Non-Functional Requirements
6	Target Architecture and Security Boundary
7	Data Model and Payment Lifecycle
8	Security Requirements and Threat Model
9	API Contract, Environments, and Integrations
10	Claude Code Workflow, Skills, and Repository Baseline
11	Implementation Plan and Demo Runbook
12	Testing, Observability, and Definition of Done
13	Risk Register, Assumptions, and Open Questions
14	Appendices

How to use this document

This document is both a BRD and a build blueprint. It can be used to align the webinar story, guide Claude Code prompts, structure the repository, and rehearse the payment demo.

1. Executive Summary

WarungKit is a compact digital-product storefront designed as a live webinar case study. The project demonstrates how a vibe-coded website becomes a trustworthy business application only after it adds real engineering boundaries: a backend API, controlled database access, payment verification, webhook handling, secret management, and security controls.

The live story is intentionally practical. Participants begin with a storefront that looks simple, choose one product, complete checkout, receive a Mayar payment link, and see the order change from pending to paid only after backend verification. The lesson is not that AI can build everything automatically; the lesson is that AI accelerates implementation when the developer still designs the system correctly.

The core business outcome

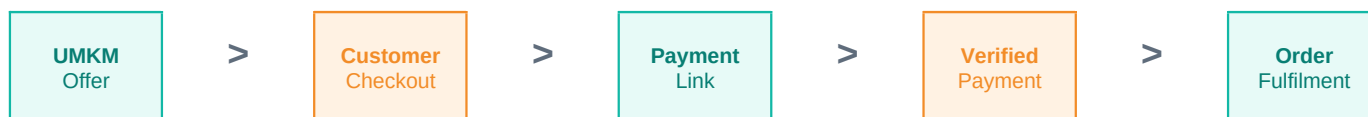


Figure 1. WarungKit end-to-end business outcome.

Key decisions locked for the webinar

Area	Decision
Product model	Digital products and a simple service. No inventory, shipping, or multi-item cart.
Frontend	React + TypeScript + Tailwind CSS deployed as a static web app.
Backend	Hono API running on Cloudflare Workers.
Database	Supabase PostgreSQL. Browser access to sensitive order records is not allowed.
Payment	Mayar invoice/payment-link flow, initiated only by backend code.
Demo environment	Public platform-provided URLs. A custom domain is not required for the webinar.
Coding workflow	Claude Code within Visual Studio Code, guided by project rules and reusable skills.

Non-negotiable design principle

The browser is an untrusted environment. It can collect checkout data and display payment status, but it must never hold payment secrets, set price, or confirm that a transaction is paid.

2. Business Context and Webinar Objective

2.1 Background

AI-assisted development has lowered the barrier to making web interfaces. Many people can now generate a landing page, a catalog, or a simple checkout experience in minutes. The gap appears when a website needs to accept money, hold customer data, react to payment events, and stay safe when exposed to the public internet.

WarungKit turns that gap into the central teaching moment. The project uses a deliberately familiar UMKM use case: selling a digital asset or a short service consultation. The audience can understand the business flow quickly, while technical participants can inspect the architecture underneath it.

2.2 Webinar objectives

ID	Objective
O-01	Explain what vibe coding is and where it is useful.
O-02	Differentiate an AI assistant from an AI-led coding workflow with project context, skills, rules, and verification.
O-03	Demonstrate a secure payment journey from checkout to verified order status.
O-04	Show why frontend-only payment integration is unsafe.
O-05	Give participants a reusable baseline for their own small business applications.

2.3 Recommended 60-minute session flow

Timing	Section	Audience Outcome
0-5 min	Context	Why a beautiful AI-generated website is not enough to take payment safely.
5-12 min	Vibe coding fundamentals	Definition, brief evolution, AI assistant vs vibe coding workflow, and human responsibility.
12-20 min	Architecture	Frontend, backend, database, payment gateway, webhooks, secrets, and data boundaries.
20-30 min	Claude Code workflow	Show project rules, skills, folder structure, and how to ask AI to plan before implementing.
30-45 min	Storefront demo	Catalog, checkout form, backend order creation, and payment-link generation.
45-55 min	Payment and security demo	Mayar payment, webhook receipt, verification, order status update, and Supabase observability.
55-60 min	Takeaways and Q&A	Checklist participants can reuse for their own project.

Narrative anchor for the session

Do not frame the demo as "AI made a website". Frame it as "AI accelerated a developer who still designed the data, security, and payment flow correctly".

3. Product Vision, Personas, and Scope

3.1 Product vision

WarungKit is a mini storefront for digital products and lightweight services that help UMKM owners, creators, and online sellers operate more effectively. The webinar version is intentionally narrow: one product per checkout, one payment flow, one reliable proof of payment.

3.2 Demo product catalog

Product	Demo Price	Type	Why It Works in Demo
Template Konten Instagram UMKM	Rp49.000	Digital product	Editable social-media content template aimed at small businesses.
Paket SOP Operasional	Rp79.000	Digital product	Operational checklist and SOP starter pack for micro-business workflows.
Sesi Konsultasi Bisnis 30 Menit	Rp149.000	Service	Short virtual consultation slot for business planning and digital operations.

3.3 Target personas

Persona	Current Need	What the Webinar Gives
UMKM owner / creator	Wants to monetize a service or downloadable asset without building a full marketplace.	Clear business flow and confidence that payments can be integrated safely.
Non-technical participant	Wants a concrete explanation without getting lost in implementation details.	Visual understanding of why each system component exists.
Developer	Wants reusable patterns for API, database, webhook, and deployment.	Implementation-level design decisions and guardrails.
Aspiring vibe coder	Can prompt AI but is unsure how to make results dependable.	A practical workflow: plan, structure, build, verify, and ship safely.

3.4 Scope boundary

Boundary	Definition
In scope	Landing page, product catalog, checkout form, backend order creation, Mayar payment link, payment status page, webhook receiver, payment verification, pending-to-paid update, Supabase observability, and security baseline.
Out of scope	Multi-item cart, customer account, full admin console, stock, delivery/logistics, refund automation, invoice PDF, WhatsApp notification, multi-merchant model, and commercial production launch.
Future extension	Secure digital delivery, customer authentication, admin analytics, notification orchestration, and custom domain.

4. User Journey and Business Rules

4.1 Primary customer journey

Step	Phase	System Behavior
1	Discover	Customer opens WarungKit and sees the product catalog.
2	Select	Customer selects one product and clicks Beli Sekarang.
3	Checkout	Customer enters name, email, and WhatsApp number.
4	Validate	Frontend validates basic usability rules; backend validates authoritative input rules.
5	Create order	Backend retrieves the product from the database and creates an order with status pending.
6	Create payment	Backend creates a payment request with Mayar and stores only the identifiers needed for reconciliation.
7	Pay	Customer is redirected to the provider-hosted payment page.
8	Receive event	Mayar sends a webhook event to the public backend endpoint.
9	Verify	Backend verifies the payment result server-side before changing the order state.
10	Complete	Customer opens payment status page and sees paid, pending, or expired status.

4.2 Business rules

ID	Rule
BR-01	One checkout contains one product only.
BR-02	The client sends a product identifier, never a final price.
BR-03	Backend retrieves the active product and price from the database.
BR-04	Every new order begins as pending.
BR-05	A browser redirect is a user-experience event, not proof of payment.
BR-06	Only verified backend logic may set order status to paid.
BR-07	Webhook processing must be idempotent. The same event must not apply twice.
BR-08	Payment secrets must never enter frontend source code, environment variables, browser traffic, or logs.
BR-09	Customer PII is limited to what is necessary for checkout and support.
BR-10	Sensitive order tables are not available directly to the public browser.
BR-11	Each checkout requires an idempotency key to avoid duplicate orders from retries or repeated clicks.

ID	Rule
BR-12	Payment events must be auditable while avoiding unnecessary raw sensitive data retention.

The live teaching moment

Show the audience that the browser can be altered by a user. Any browser-supplied price, paid status, or secret must be treated as untrusted. The backend is where business truth is enforced.

5. Functional and Non-Functional Requirements

5.1 Functional requirements

ID	Feature	Acceptance Criteria
FR-01	Product Catalog	Three active demo products appear with name, description, price, and CTA.
FR-02	Checkout Form	Customer must enter name, email, and WhatsApp number before checkout.
FR-03	Validation	Invalid or incomplete input receives clear feedback. Backend validates independently.
FR-04	Order Creation	System creates a unique order code and initial pending record.
FR-05	Price Resolution	Backend resolves product and price from database; no final price accepted from client.
FR-06	Payment Link	Backend creates payment request and returns a redirect-safe payment URL.
FR-07	Payment Redirect	Customer reaches the payment provider page after checkout.
FR-08	Webhook Receiver	Worker receives payment event at a stable public route.
FR-09	Payment Verification	Backend verifies provider state before changing order to paid.
FR-10	Payment Status	Customer sees pending, paid, expired, failed, or cancelled outcome.
FR-11	Idempotency	Duplicate checkout and webhook requests do not create duplicate orders or state changes.
FR-12	Demo Observability	Order and payment-event state can be shown safely in Supabase during rehearsal/demo.

5.2 Non-functional requirements

Area	Requirement
Security	Secrets exist only in server-side secure configuration. No payment secret or database service key appears in browser.
Privacy	Collect minimum customer PII. Use masked or absent PII in logs.
Reliability	Webhook handler is idempotent, retry-tolerant, and does not mark paid when verification fails.
Performance	Target checkout API response under 3 seconds excluding payment completion.
Scalability	Frontend and API remain stateless. Database is the source of order state.
Maintainability	UI, routes, services, repositories, contracts, and migrations are separated.
Observability	Every request has request ID. Payment processing exposes safe status logs and event record.
Demo Readiness	Primary flow is smoke-tested before event; backup media or state is prepared.
Accessibility	Form labels, visible error messages, keyboard focus, and basic mobile responsiveness are included.

6. Target Architecture and Security Boundary

6.1 Architecture overview

WarungKit follows a browser-to-backend-to-provider model. The frontend is responsible for presentation and basic interaction. The Worker API is responsible for enforcing business rules, calling external APIs, and writing to the database. Payment provider events return to the Worker, which verifies them before updating the order.

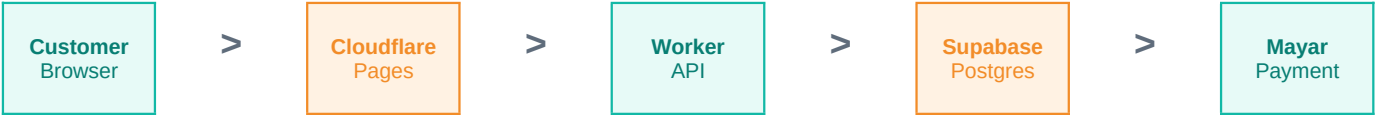


Figure 2. Logical component map. The Worker API is the security boundary between browser, database, and payment provider.

6.2 Component responsibilities

Component	Responsibility
Customer Browser	Displays catalog, captures checkout data, starts checkout, polls/reads payment status. Never owns business truth.
Cloudflare Pages	Hosts static frontend artifacts and provides public web entry point.
Cloudflare Worker API	Validates requests, resolves product price, creates order, calls Mayar, processes webhook, verifies status, updates orders, applies CORS, request IDs, rate-limit readiness, and safe errors.
Supabase PostgreSQL	Stores products, orders, payment event metadata, and idempotency records. RLS enabled; service access only from Worker.
Mayar	Hosts payment page, receives payment, and issues payment-related events/identifiers used for server-side verification.
Claude Code	Accelerates planning and implementation, but does not replace review, testing, or security ownership.

6.3 Trust boundary

Boundary	What It Means
Untrusted zone	Customer browser, browser storage, incoming form data, redirect parameters, and public internet requests.
Controlled API zone	Cloudflare Worker validates all client input and only exposes public-safe responses.
Protected data zone	Supabase service access, payment keys, and order mutation logic.
External provider zone	Mayar responses and webhook payloads are treated as external input until verified.

6.4 Secure payment flow

Step	Processing Rule
A	Frontend sends productId, customer details, and idempotency key to POST /api/checkout.
B	Worker validates input and retrieves canonical product and price from database.

Step	Processing Rule
C	Worker writes pending order and calls Mayar using a server-side secret.
D	Worker stores payment identifier and returns payment URL to browser.
E	Customer pays on provider-hosted payment page.
F	Provider webhook calls POST /api/webhooks/mayar.
G	Worker records event safely, verifies payment status server-side, and performs idempotent state transition.
H	Order becomes paid only after verification. Status page reflects database state.

Security boundary in one sentence

Frontend requests intent. Backend decides truth. The database records truth. The payment provider is verified before payment truth changes.

7. Data Model and Payment Lifecycle

7.1 Core entities

Entity	Purpose	Core Fields
products	Catalog source of truth	id, slug, name, description, price_idr, product_type, is_active, sort_order, created_at, updated_at
orders	Customer checkout and payment state	id, order_code, product_id, customer_name, customer_email, customer_phone, amount_idr, status, mayar_invoice_id, mayar_invoice_url, receipt_token, paid_at, metadata, timestamps
payment_events	Idempotent payment event audit	id, provider, provider_event_hash, provider_event_id, sanitized_payload, processing_status, processed_at, order_id, timestamps
checkout_idempotency	Checkout replay protection	id, idempotency_key, request_hash, order_id, response_payload, expires_at, timestamps

7.2 Relationship model

```

products (1) ----- (many) orders
orders (1) ----- (many) payment_events
orders (1) ----- (many) checkout_idempotency

Source of truth for amount: products.price_idr
Source of truth for payment state: orders.status
Source of truth for replay prevention: payment_events + checkout_idempotency

```

7.3 Suggested order lifecycle

Status	Meaning
draft	Optional transient state before database persistence. Not required in live demo.
pending	Order is created and waiting for payment request or payment completion.
payment_created	Provider payment identifier and payment URL are stored.
paid	Payment is verified server-side and fulfilment may begin.
expired	Payment request expires before successful verification.
failed	Payment attempt or verification fails.
cancelled	Checkout is deliberately cancelled or abandoned by business logic.

7.4 Allowed transitions

From	To	Trigger
pending	payment_created	Payment request successfully created.
payment_created	paid	Webhook/verification confirms payment.
payment_created	expired	Provider reports expiry or internal reconciliation detects timeout.

From	To	Trigger
payment_created	failed	Verification or payment fails.
pending	cancelled	Checkout aborted before payment link becomes usable.
paid	paid	Duplicate webhook or retry. No side effect except safe audit entry.

Data minimisation rule

Store only what the demo needs to process checkout and prove the flow. Customer name, email, and WhatsApp number are enough. Never store card data, bank credentials, payment secrets, or user passwords.

8. Security Requirements and Threat Model

8.1 Mandatory security controls

ID	Control	Requirement
SC-01	Secrets	Mayar key and Supabase service role key exist only in Worker secret/configuration.
SC-02	Client trust	Client price, paid status, redirect query, and browser storage are not treated as authoritative.
SC-03	Input validation	Validate schema at frontend for UX and independently at backend for enforcement.
SC-04	Database access	All sensitive database access runs through Worker repository layer. RLS is enabled on all tables.
SC-05	Webhook handling	Use event hashing/deduplication; verify payment result server-side before paid transition.
SC-06	CORS	Allow only declared frontend origins. Do not use broad permissive policy in final demo.
SC-07	Logging	Use request IDs. Mask PII and never log secrets or full sensitive payment payload.
SC-08	Rate limiting	Design route-level readiness for checkout and webhook abuse protection.
SC-09	Error handling	Return safe messages to client; keep technical detail in controlled logs only.
SC-10	Data retention	Set a demo retention decision for customer PII and payment event records.

8.2 Threat model

Threat	Example Attack	Mitigation
Leaked payment key	Key is accidentally committed or placed in frontend.	No secrets in source; .gitignore; Worker secrets; secret scanning awareness; code review.
Price manipulation	User edits price in browser dev tools.	Client sends productId only; backend resolves product price from database.
Fake paid redirect	User opens success URL manually.	Status page reads backend/database state; only verified Worker update sets paid.
Webhook replay	Same provider event is delivered repeatedly.	Store provider event hash/id; idempotent update; paid-to-paid is no-op.
Webhook spoofing	Attacker sends request to webhook route.	Validate expected provider event pattern, verify payment state using provider server-side API, log anomaly.
Duplicate checkout	Double-click/retry creates two orders.	Idempotency key, request hash, and response replay behavior.
Direct database access	Browser accesses sensitive table/data.	No service key in browser; restrictive RLS; API-only mutation path.
PII overexposure	Customer data appears in logs/screenshots.	Minimal collection, masking, controlled demo account, clean screen-sharing discipline.
Unsafe AI code generation	Claude generates insecure shortcut.	CLAUDE.md, scoped skills, tests, code review, and no automatic trust.

Threat	Example Attack	Mitigation
Misconfigured CORS	Unexpected origin calls public API.	Explicit allowlist driven by environment variable.

8.3 Security checklist for live demo

- No secret dashboard or key value is ever shown on screen.
- The checkout request in browser network panel contains no Mayar or Supabase service key.
- Only a product identifier is sent to checkout; client does not send final amount as a trusted field.
- Payment success page is demonstrated separately from backend order truth.
- At least one duplicate webhook scenario is explained or testable.
- Supabase table view is opened only with non-sensitive demo records.
- Repo has no .env or .dev.vars secrets committed.
- CORS origin is locked to the demo web URL before rehearsal.

9. API Contract, Environments, and Integrations

9.1 Public API surface

Method	Route	Purpose	Notes
GET	/health	Public health check	Returns service health only.
GET	/api/products	Catalog query	Returns active public product data.
POST	/api/checkout	Create checkout	Validates request, creates/reuses order, initiates payment link.
GET	/api/orders/:orderId	Payment status	Returns safe order status data. Restrict by receipt token or equivalent public-safe mechanism.
POST	/api/webhooks/mayar	Provider event receiver	Records, validates, verifies, and idempotently reconciles payment state.

9.2 Checkout contract - high level

POST /api/checkout

Request

```
{
  "productId": "uuid-or-public-product-id",
  "customer": {
    "name": "Customer Name",
    "email": "customer@example.com",
    "phone": "+628xxxxxxxxxx"
  },
  "idempotencyKey": "client-generated-uuid"
}
```

Response

```
{
  "orderId": "server-generated-id",
  "orderCode": "WK-...",
  "status": "payment_created",
  "paymentUrl": "provider-hosted-url",
  "expiresAt": "ISO-8601 timestamp"
}
```

9.3 Environment variables

Variable	Value Category	Purpose
ENVIRONMENT	development staging demo	Behavior and safe logging context.
ALLOWED_ORIGINS	https://warungkit-demo.[platform-url]	CORS allowlist.
SUPABASE_URL	server-side only	Supabase project URL.
SUPABASE_SERVICE_ROLE_KEY	server-side secret	Privileged database access from Worker only.
MAYAR_API_KEY	server-side secret	Payment API credential.
MAYAR_API_BASE_URL	provider base URL	Environment-specific provider API configuration.
APP_PUBLIC_URL	frontend public URL	Return/callback and public status link construction.

Variable	Value Category	Purpose
WEBHOOK_VERIFICATION_MODE	strict demo	Operational mode for controlled rehearsal only.

9.4 Environment strategy

Environment	Purpose	Rules
Local	Developer machine	Use local frontend/API and non-production test credentials. Never expose local URL to payment webhook unless a temporary public tunnel is intentionally configured.
Demo	Public deployment for webinar rehearsal and event	Stable public URLs, controlled test products, restricted CORS, secret config, and expected webhook endpoint.
Production future	Commercial launch only after scope expansion	Custom domain, monitoring, formal data retention, access reviews, incident process, and stronger security testing.

Integration contract rule

Do not assume exact provider request or webhook payload fields in the BRD. Map them during implementation against the current Mayar documentation and keep the provider client isolated in one service layer.

10. Claude Code Workflow, Skills, and Repository Baseline

10.1 AI-assisted development operating model

Claude Code is used inside Visual Studio Code as an implementation partner. It should receive a clear project contract, a bounded task, required inputs, acceptance criteria, and verification commands. It should not be asked to invent business rules, expose secrets, or make uncontrolled project-wide changes.

Stage	Working Rule
1. Plan	Ask Claude to inspect the relevant document, propose a plan, and identify assumptions before editing.
2. Scope	Reference one skill and one feature area. Keep prompts narrow enough to review.
3. Implement	Change code only within intended folders. Use contracts, validation, and error handling.
4. Verify	Run lint, typecheck, tests, and build. Perform a manual smoke test for payment flow.
5. Commit	Commit at meaningful checkpoints that match the demo storyline.

10.2 Required Claude Code project files

File	Purpose
CLAUDE.md	Permanent project rules, architecture guardrails, and validation expectations.
.claude/skills/frontend-design/SKILL.md	Responsive UI, accessibility, layout, and frontend quality rules.
.claude/skills/backend-api/SKILL.md	Route structure, request validation, safe errors, middleware, and API tests.
.claude/skills/database-engineering/SKILL.md	Migration-first schema changes, repository rules, indexes, and RLS.
.claude/skills/payment-integration/SKILL.md	Provider adapter, idempotency, webhook verification, and reconciliation.
.claude/skills/security-review/SKILL.md	Secret handling, CORS, PII, abuse cases, and live-demo checks.
.claude/skills/testing-qa/SKILL.md	Unit/integration/manual smoke test rules and test data discipline.
.claude/skills/demo-runbook/SKILL.md	Checkpoint branches, rehearsal steps, fallback plan, and screen-sharing safety.

10.3 Production-aware repository structure

warungkit/	
├── apps/	
│ ├── web/	# React storefront
│ └── api/	# Hono Worker API
├── packages/	
│ └── contracts/	# Shared Zod schemas and TypeScript types
├── supabase/	
│ ├── migrations/	# Versioned schema and RLS rules
│ └── seed.sql	# Demo products only
├── .claude/	
│ └── skills/	# Scoped Claude Code skills
└── docs/	
│ ├── BRD.md	
│ ├── architecture.md	
│ ├── threat-model.md	
│ └── security-checklist.md	

```
|
| └─ api-contract.md
| └─ demo-runbook.md
| └─ decisions/
└─ scripts/                # Local helper scripts, no secrets
└─ tests/                  # Cross-app/integration tests if needed
└─ .github/workflows/      # CI only, no auto-deploy initially
└─ CLAUDE.md
└─ AGENTS.md               # Optional tool-neutral collaboration rules
└─ package.json
└─ pnpm-workspace.yaml
```

10.4 Non-negotiable CLAUDE.md rules

- Use strict TypeScript. Do not use any unless explicitly justified and reviewed.
- Never place secrets in source code, frontend configuration, snapshots, or tests.
- All database writes go through repository layer; schema changes require a migration.
- All payment calls go through a dedicated service/adaptor layer.
- Validate every external input and return safe client-facing errors.
- Never mark an order paid from a browser redirect alone.
- Webhook processing must be idempotent and must verify provider state server-side.
- Before declaring work complete, run lint, typecheck, tests, and build.
- Do not create automatic deployment workflow until the demo is validated manually.

11. Implementation Plan and Demo Runbook

11.1 Build phases

Phase	Name	Deliverable
Phase 0	Document baseline	BRD, architecture decision, threat model, API contract, demo story.
Phase 1	Repository baseline	Monorepo, app skeletons, shared contracts, CI, .gitignore, Claude skills.
Phase 2	Database foundation	Supabase project binding, migrations, RLS, seed product catalog.
Phase 3	Storefront UI	Catalog, product cards, checkout form, basic accessible states.
Phase 4	Secure checkout API	Validation, product lookup, order creation, idempotency, error model.
Phase 5	Payment integration	Mayar adapter, invoice creation, payment URL, safe response.
Phase 6	Webhook and verification	Webhook endpoint, event deduplication, provider verification, state transition.
Phase 7	Rehearsal and hardening	Smoke test, fallback assets, logs, CORS, secret review, demo branch checkpoints.

11.2 Git demo checkpoints

Checkpoint	Expected State
demo-start	UI and product catalog work. Checkout can be shown, but it is still a controlled dummy state.
demo-payment	Backend creates a real test payment request and returns a payment URL.
demo-final	Webhook and server-side verification update order from pending/payment_created to paid.
demo-backup	Known-good screenshots, screen recording, and pre-created successful test transaction.

11.3 Rehearsal timeline

When	Action
H-7 days	Lock project names, public demo URLs, and non-secret configuration template.
H-5 days	Complete product catalog and basic checkout. Confirm database migrations and demo products.
H-3 days	Integrate payment test environment. Verify payment-link creation and status query.
H-2 days	Verify webhook delivery, duplicate-event handling, order state transition, CORS, and secret placement.
H-1 day	Run full 60-minute dry run. Record a backup happy-path screen capture.

When	Action
H-0	Use stable public URLs. Open only pre-verified tabs. Start with demo-start branch and move through planned checkpoints.

11.4 Live demo operator checklist

- Use a dedicated demo browser profile with no personal credentials or notifications.
- Open frontend, Worker logs, Supabase order table, and payment provider test page in separate tabs before session.
- Do not show secrets, environment variable values, or personal data.
- Start from a clean pending order or clear test fixture.
- Explain before clicking: what the frontend does, what the backend does, and what the payment provider does.
- After payment, show the order state in database only after verified webhook processing.
- Keep backup screenshot/video in an accessible folder but do not rely on it unless needed.

12. Testing, Observability, and Definition of Done

12.1 Test layers

Layer	Example Coverage	Purpose
Static validation	pnpm lint, pnpm typecheck, pnpm build	Checks style, types, and build integrity.
Unit tests	Zod schemas, amount resolution, idempotency helper, order state transition	Validates isolated business rules.
API integration tests	Health, products, checkout validation, duplicate checkout behavior	Validates request/response behavior.
Webhook tests	Duplicate event, invalid event, verification failure, paid transition	Validates reliability and security logic.
Manual smoke test	Browser checkout through provider test payment and order result	Validates real user journey.
Demo rehearsal	Full narrative, tabs, timing, fallback content	Validates human/operator experience.

12.2 Safe observability

Signal	Usage
Request ID	Generated at API entry and returned in safe error responses.
Order Code	Human-readable identifier suitable for demo reconciliation.
Payment Event Hash	Used to detect replay without relying on raw payload display.
Structured Logs	Use event name, state transition, request ID, and masked identifiers.
Dashboard View	Use Supabase only to show controlled demo rows and status fields.

12.3 Definition of done for demo-ready status

- Frontend loads successfully from public demo URL.
- Product catalog is sourced from tested demo data.
- Checkout validates required fields and shows usable error messages.
- Backend creates one order for one idempotency key.
- Payment request is created only from backend and returns valid provider payment URL.
- No secret appears in frontend bundle, browser network request, git history, or screen-share flow.
- Webhook delivery has been tested against stable public endpoint.
- Order becomes paid only after server-side verification path succeeds.
- Duplicate webhook does not create duplicate state change or duplicate fulfilment action.
- CORS allowlist is confirmed against demo frontend URL.
- All lint, typecheck, test, and build commands pass.
- Full rehearsal and backup flow are ready.

13. Risk Register, Assumptions, and Open Questions

13.1 Risk register

ID	Risk	Likelihood	Impact	Mitigation
R-01	Internet instability	Medium	High	Mobile hotspot, backup recording, screenshots, and pre-opened local documentation.
R-02	Payment test issue	Medium	High	Rehearse provider test environment, retain successful prior transaction, and prepare fallback narrative.
R-03	Webhook delay	Medium	Medium	Demonstrate pending state first, retry/status check, then show verified event record.
R-04	Secret exposure on screen	Low	High	Use preconfigured environment, avoid dashboard secret tabs, and separate demo profile.
R-05	AI-generated unsafe shortcut	Medium	High	Use CLAUDE.md, skill boundaries, code review, and test gates.
R-06	Scope creep	High	Medium	Keep multi-cart, login, shipping, and notification out of webinar MVP.
R-07	URL/config changes close to event	Medium	High	Lock public app/API URLs and retest CORS/webhook after any deployment configuration change.

13.2 Assumptions

- Cloudflare, Supabase, GitHub, and Mayar accounts are available before implementation begins. A suitable Mayar test/sandbox workflow is available for rehearsal and live demonstration. The repository is prepared in advance; the live session focuses on meaningful code and architecture, not raw UI construction from zero. A custom domain is not required; stable public project URLs can be used for the demo. The webinar version is a controlled demonstration, not an immediate commercial go-live application. A demo data retention decision will be applied after the event, including deleting or anonymising test customer data.

13.3 Open questions

ID	Question
Q-01	Which Mayar test payment method will be used live, and what is its expected confirmation behavior?
Q-02	Will payment status page use polling, manual refresh, or an explicit check-status button?
Q-03	Will the demo show a secure digital-download placeholder after paid status, or stop at payment confirmation?
Q-04	Will the admin view be a protected WarungKit page or a controlled Supabase dashboard demonstration only?
Q-05	What exact public project names will be locked for frontend and Worker API?
Q-06	What is the data deletion date for customer PII gathered during rehearsal and event demonstration?

14. Appendices

Appendix A - Initial backlog

ID	Backlog Item	Area	Priority
B-01	Create BRD and architecture documents	Documentation	P0
B-02	Create monorepo baseline and Claude Code rules	Foundation	P0
B-03	Create Supabase migration and seed products	Database	P0
B-04	Create public product catalog	Frontend	P0
B-05	Create checkout form and client validation	Frontend	P0
B-06	Create checkout API with order/idempotency logic	Backend	P0
B-07	Create Mayar payment service adapter	Payment	P0
B-08	Create webhook receiver and verification flow	Payment	P0
B-09	Create payment status page	Frontend	P1
B-10	Create secure order-status access mechanism	Security	P1
B-11	Add unit/integration tests and CI	Quality	P0
B-12	Create rehearsal checklist and backup media	Demo	P0

Appendix B - Demo branch and command checklist

```
# Expected checkpoints
git checkout demo-start
git checkout demo-payment
git checkout demo-final

# Expected quality gate
pnpm install
pnpm lint
pnpm typecheck
pnpm test
pnpm build

# Rule: do not switch branches or deploy new code during live webinar
# unless the change has already been rehearsed.
```

Appendix C - Core messages for the final slide

- Vibe coding accelerates delivery, but it does not remove engineering responsibility. A payment button is not payment integration until the backend verifies the result. Keep secrets, price logic, and order state away from the browser. Use AI with skills, rules, testing, and checkpoints - not blind trust. Start small: one product, one checkout, one payment flow, one solid security boundary.

Approved next move: Use this document as the source of truth. The next artifact should be the Architecture Decision Record and CLAUDE.md baseline, followed by repository scaffolding and Supabase migration design.